
Cambios propuestos al Modelado de Problema

Resumen de observaciones nuevas al proyecto:

OJO: No se pueden fraccionar los datos de la columna?

EJ: Si tengo la palabra "Papel" y solo me quedan 2 bytes, no puedo fraccionar (pa)->(pel), osea poner el restante en otro sector.

VERIFICAR QUE NO SE CREE UN NUEVO ÁRBOL POR CADA CONSULTA,

1. Requerimientos Funcionales — Modificaciones

Configuración Paramétrica del Hardware (*sin cambios*): El sistema debe recibir los parámetros físicos iniciales del disco duro virtual: Número de Platos, Número de Pistas por superficie, Número de Sectores por pista y Capacidad fija en bytes por Sector (ej. 512 bytes).

Definición de Esquema (DDL Dinámico) — Actualizado: Se amplían los tipos de datos soportados para incluir tipos de longitud variable:

- **INT** (4 bytes), **FLOAT** (4 bytes), **CHAR(N)** (N bytes) — tipos existentes
- **TEXT(N)** (hasta N bytes) — nuevo tipo para cadenas largas
- **BLOB(N)** — nuevo tipo para datos binarios de gran tamaño

El motor calculará el tamaño del registro al momento de definir la tabla. Si el registro supera el tamaño de un sector, activará automáticamente el modo **multi-sector**. (spanning)

3. Estrategia de Direccionamiento Físico — Sección 3.4 Reemplazada

3.4 Manejo de registros que ocupan múltiples sectores (Spanning)

Se eliminan los 3 escenarios anteriores y se reemplaza por esta estrategia:

Clasificación de registros:

Caso	Condición	Estrategia
Registro pequeño	tamaño < sector	Empaquetado normal, fragmentación interna aceptada
Registro exacto	tamaño = sector	Ocupa exactamente un sector

Registro grande $\text{tamaño} > \text{sector}$ **Spanning multi-sector**

Estructura del encabezado de sector (Sector Header):

Cada sector reserva los primeros bytes para un pequeño encabezado de control:

None

```
[FLAG: 1 byte][RECORD_ID: 4 bytes][NEXT_SECTOR_LBA: 4 bytes][DATA: resto]
```

- **FLAG**: indica si el sector es **INICIO**, **CONTINUACIÓN** o **FIN** de un registro grande
- **RECORD_ID**: identificador lógico del registro al que pertenece
- **NEXT_SECTOR_LBA**: apunta al siguiente sector de la cadena (-1 si es el último)

Algoritmo de inserción para registros grandes:

None

1. Calcular cuántos sectores necesita: $\text{ceil}(\text{tamaño_registro} / \text{bytes_útiles_por_sector})$
2. Buscar en el Bitmap N sectores libres consecutivos (o no consecutivos si se usa encadenamiento)
3. Escribir el primer fragmento con **FLAG = INICIO**
4. Por cada sector adicional: **FLAG = CONTINUACIÓN**, encadenar con **NEXT_SECTOR_LBA**
5. Último sector: **FLAG = FIN**, **NEXT_SECTOR_LBA = -1**
6. Registrar en el Bitmap todos los sectores utilizados como OCUPADOS

4. Estrategia de Almacenamiento (Integridad de datos) — Actualizada

Se distinguen dos modos de almacenamiento:

Modo A — Registro compacto ($\text{tamaño} \leq \text{bytes_útiles_por_sector}$): igual al diseño original, múltiples registros por sector.

Modo B — Registro extendido ($\text{tamaño} > \text{bytes_útiles_por_sector}$): un registro se distribuye en una cadena de sectores enlazados. El **RecordID** apunta siempre al **sector inicial** de la cadena.

OJO: es importante que cada dato deba guardarse de manera integra. EJ. si tengo 2 bytes restantes en un sector, y quiero guardar un INT, NO se puede guardar 2 bytes en el sector actual y otros 2 bytes en el siguiente sector.

5. Estrategia de Búsqueda y creación del Árbol— Actualizada

El nodo del Árbol AVL ahora almacena adicionalmente el campo `num_sectores` dentro del `RecordID`, para que al recuperar un registro grande el motor sepa cuántos accesos a disco realizará.

- **Búsqueda exacta:** $O(\log n)$ en RAM + `ceil(tamaño / bytes_útiles)` accesos a disco
- **Métrica I/O:** se actualiza para reportar los sectores leídos de la cadena, no solo 1

El árbol AVL se tiene que armar de inicio, de manera que al momento de añadir una fila, se indexa cada atributo al árbol. Otra observación, se debe mostrar el recorrido de búsqueda, ej. buscas en un rango de 10-20, deben salir los nodos de ahí.

Se genera un árbol por columna, pero uno nuevo por consulta no.

6. Diseño de la Interfaz — Cambio en Panel B

Panel B actualizado: Cada fila de resultado mostrará, para registros grandes, la cadena de sectores utilizada:

None

```
[Plato: U, Cara: V, Pista: W, Sector_Inicio: X → Sector_N: Y, Bytes: Z]
```

Estrategia de Validación por Capas

Se propone un sistema de validación en **3 momentos distintos**, antes de tocar el disco:

Capa 1 — Al definir el esquema (DDL)

Cuando el usuario define los campos de la tabla, el motor calcula inmediatamente el tamaño del registro y lo compara contra la capacidad total:

None

```
capacidad_total = P × 2 × T × S × bytes_útiles_por_sector
```

```
Si tamaño_registro > capacidad_total:
```

```
    → ERROR en DDL: "El esquema definido supera la capacidad  
total del disco.
```

```
                                Reduzca los campos o reconfigure el  
hardware."
```

El sistema **bloquea la creación de la tabla** y no permite continuar.

Capa 2 — Al insertar un registro individual

Antes de escribir, el motor consulta el Bitmap para contar sectores libres disponibles:

None

```
sectores_necesarios = ceil(tamaño_registro /  
bytes_útiles_por_sector)  
sectores_libres      = contar_ceros(Bitmap)
```

```
Si sectores_necesarios > sectores_libres:
```

```
    → ERROR en INSERT: "Espacio insuficiente en disco.  
                        Sectores requeridos: X | Sectores libres:  
Y"
```

No se escribe nada, el Bitmap no se modifica.

Capa 3 — Al cargar un CSV masivo

Antes de iniciar la carga del archivo completo, el motor hace una **validación previa global**:

None

```
registros_en_csv      = contar_filas_del_CSV  
sectores_necesarios = registros_en_csv × ceil(tamaño_registro /  
bytes_útiles)  
sectores_libres       = contar_ceros(Bitmap)
```

```
Si sectores_necesarios > sectores_libres:
```

```
    → ADVERTENCIA: "El archivo contiene N registros pero el disco  
solo
```

tiene capacidad para M. ¿Cargar
parcialmente?"

Aquí se le ofrecen **dos opciones al usuario**:

- ☒ Cargar solo hasta llenar el disco (carga parcial)
- ☐ Cancelar e ir a reconfigurar el hardware

Respuesta en la Interfaz (Panel B y Panel A)

Panel	Comportamiento
Panel A	Muestra en tiempo real los sectores libres restantes mientras se configura el hardware
Panel B	Al definir campos, muestra "Capacidad usada del disco: X%" con barra visual
Panel B	Al insertar, si falla, resalta en rojo el campo de estado del registro con el error específico

Impacto en el AVL

Si una inserción falla por falta de espacio, el nodo **no se agrega al AVL**, garantizando consistencia entre el índice en RAM y los datos físicos en disco. El árbol solo refleja registros efectivamente escritos.